# Learning Path From a Pile of Docs

### Find the docs that depend on each other, group them, then order everything

**Tracks:** Graphs (strongly connected components, Tarjan, Kosaraju, condensation DAG, topological sort)
**Reading time:** about 30 minutes, longer if you try it yourself first **Code:** `src/LearningPath/`. **Tests:** 65, run with `dotnet test`. **Benchmark:** `benchmarks/`

---

## 0. The problem

> "We have a pile of docs. Each one lists the docs you should read before it. Give me a reading order where every doc comes after the docs it needs. The docs were written by different people over time, so some of them refer to each other. That can't be an error, we still need an order."

**Try it before you read on.** Hints, if you need them:

▼ Hint 1
Without the cycles this is a plain topological sort. What does Kahn's algorithm do when two docs each wait for the other?

▼ Hint 2
If A needs B and B needs A, there is no "first" between them. What if you treat them as one unit?

▼ Hint 3
Two docs belong in the same unit exactly when each one can reach the other. Can one DFS find all of those groups?

---

## 1. Why this problem exists

The agent plan executor in this repo runs a plan with Kahn's algorithm. Kahn's needs a graph with no cycles. If there is one, it can tell you a cycle exists (some nodes never get placed), but it can't order them. For an agent plan that's fine: a plan with a cycle is a bad plan, so we reject it.

A pile of docs is different. People write docs that point at each other all the time, and rejecting the whole pile isn't an option. So we need one more step before Kahn's:

1. Find every group of docs that depend on each other in a circle. These are the **strongly connected components** (SCCs).
2. Squash each group into a single node. The result is the **condensation**, and it never has a cycle.
3. Order the condensation with Kahn's.

### Where I use it: building an agent's context

In the hiring assistant I've been using as the example in this repo, the agent loads a set of instruction docs before it screens a candidate: the job spec format, the scoring rubric, the candidate dossier format, the

interview loop and so on. The code that builds the prompt has to decide the order:

- **Prerequisites first.** The job spec format goes in before the rubric that uses it.
- **Cycles together.** The scoring rubric is defined against the dossier's fields, and the dossier says which rubric scores it carries. Neither comes first, so they go in side by side.
- **The same order every time.** If the order changes between calls, the prompt beginning changes, and the prefix cache can't reuse anything. The order here only depends on the docs themselves, not on the order they were handed in. A test checks that.
- **A list of the tangles.** The cycle groups are useful on their own: they tell whoever owns the docs where the circular references are.

### The same pipeline outside AI

- **CMake** runs Tarjan's algorithm on target dependencies, allows a cycle only if every target in it is a static library, and then uses the DAG of components to build a safe order (`cmComputeTargetDepends.cxx`, `cmComputeComponentGraph.h`). That's this problem, with libraries instead of docs.
- **LLVM** walks the SCCs of a graph with Tarjan's algorithm, and visits a component only after the components it points to (`SCCIterator.h`).
- **Haskell's `containers` library** has `stronglyConnComp`, which returns the SCCs of a graph "reverse topologically sorted" (`Data.Graph`).

---

## 2. Questions that shape the design (and the answers assumed here)

| Ask | Why it changes the design | Assume |
|---|---|---|
| Which way do edges point? | Everything else depends on it | From the prerequisite to the doc that needs it: "read u before v" |
| Docs in a cycle: error or group? | Rejecting is what the agent plan executor does | **Group** them and read them together |
| A doc lists a doc that isn't in the pile | Silent skip hides broken links | **Reject**, and report every missing one at once |
| A doc lists itself | A self-loop is a cycle of one | Keep it, and mark its group as a cycle |
| The same prerequisite listed twice | Inflates in-degrees in Kahn's | Count it once |
| Several docs ready at the same time | Any choice is valid, but then the order isn't stable | Smallest doc id first (ordinal), so the same pile always gives the same path |
| How big can the pile get? | Recursive DFS puts one stack frame per doc on the thread stack | **Iterative** DFS, tested on a 1,000,000-doc chain |

---

## 3. The mental model

Two docs are in the same group exactly when each one can reach the other. That's what an SCC is.

Once every group is squashed to one node, there can't be a cycle left: a cycle between groups would make all of them reach each other, so they'd have been one group in the first place. And a graph with no cycles is exactly what Kahn's can order.

So the work is:

1. Tarjan finds the groups, in one DFS.
2. The condensation builds one node per group and one edge per pair of groups that have an edge between them.
3. Kahn's orders the groups, breaking ties by the smallest doc id in each group.

So Kahn's is still there. Tarjan runs before it.

---

## 4. Tarjan's algorithm

Run a DFS and give every node a number when you first reach it. That's its **index**. Every node also keeps a **low-link**: the smallest index it can reach from inside its DFS subtree, using at most one edge back to a node whose group isn't finished yet.

Keep a second stack, the **component stack**, holding the visited nodes whose group isn't known yet.

For each node `v`:

1. Set `index[v] = low[v] = next number`, and push `v` on the component stack.
2. For each edge `v` to `w`:
   - If `w` is not visited, visit it, then `low[v] = min(low[v], low[w])`.
   - If `w` is on the component stack, `low[v] = min(low[v], index[w])`.
   - Otherwise `w`'s group is already finished. It can't be part of `v`'s group, so ignore the edge.
3. When all edges are done and `low[v] == index[v]`, nothing below `v` climbs above it. `v` is the root of a group: pop the component stack down to `v`, and those nodes are the group.

### A run on the hiring assistant's docs

The pile (each doc and what it says to read first):

| Doc | Read first |
|---|---|
| agent prompt | tool permissions, job spec format |
| candidate dossier | scoring rubric, job spec format |
| interview loop | scoring rubric, candidate dossier, job spec format |
| job spec format | nothing |
| offer decision | interview loop, scoring rubric |
| scoring rubric | job spec format, candidate dossier |
| tool permissions | nothing |

Docs are numbered in sorted order, and the DFS starts from each unvisited doc in that order. Edges point from the prerequisite to the doc that needs it.

| Step | What happens | Component stack | Groups so far |
|---|---|---|---|
| 1 | Visit agent prompt, index 0. It's nobody's prerequisite, so no edges. low = index | agent prompt | |
| 2 | Pop it: a group of one | | {agent prompt} |
| 3 | Visit candidate dossier, index 1. Edge to interview loop | dossier | |
| 4 | Visit interview loop, index 2. Edge to offer decision | dossier, interview | |
| 5 | Visit offer decision, index 3. No edges, low = index, pop it | dossier, interview | + {offer decision} |
| 6 | Back in interview loop: low stays 2, equal to its index, pop it | dossier | + {interview loop} |
| 7 | Back in dossier. Edge to scoring rubric: visit it, index 4 | dossier, rubric | |
| 8 | Rubric has an edge back to dossier, which is still on the stack: low[rubric] = 1 | dossier, rubric | |
| 9 | Rubric's other edges go to finished groups, ignored. Rubric is done, low 1 is not its index 4, so it stays on the stack. Dossier takes low 1 | dossier, rubric | |
| 10 | Dossier is done with low = index = 1: pop down to dossier | | + {candidate dossier, scoring rubric} |
| 11 | Visit job spec format, index 5. All its edges go to finished groups | | + {job spec format} |
| 12 | Visit tool permissions, index 6. Same | | + {tool permissions} |

Six groups, and one of them is a cycle. Notice the order they came out in: agent prompt, offer decision, interview loop, the rubric and dossier group, job spec format, tool permissions. Every group came out after everything that depends on it. Tarjan always does this: the groups come out in **reverse topological order**.

---

## 5. Why it's correct

**One group never splits across two pops.** The first node of a group that the DFS reaches has the smallest index in that group. Every other node of the group is reached from it, so they all sit above it on the component stack. None of them can be popped earlier, because each of them can climb back to that first node, so its low-link is smaller than its own index.

**A pop never takes too much.** When `low[v] == index[v]`, nothing in `v`'s subtree reaches a node that's below `v` on the stack and still open. Any node above `v` on the stack is reached from `v`, and it hasn't been popped, so it reaches back to `v` or higher. So everything popped reaches `v` and is reached from `v`, which is exactly one group.

**Ignoring edges to finished groups is safe.** If `w`'s group is finished and `v` is still open, then `w` can't reach `v` (otherwise `v` would have been in `w`'s subtree and popped with it). So `v` and `w` can't be in the same group.

The condensation is acyclic by definition (section 3), so Kahn's always places every group. The code still checks it and throws if it doesn't, because that would mean a bug in the SCC step.

---

## 6. From groups to a reading order

Tarjan already gives a valid order if you read its groups backwards. I still run Kahn's on the condensation, for one reason: I want to choose between groups that are ready at the same time.

When several groups are ready, the one with the smallest doc id goes first. With the hiring docs:

1. Ready at the start: job spec format and tool permissions. "job spec format" is smaller, so it goes first.
2. Now the rubric and dossier group is ready. Its smallest member is "candidate dossier", which is smaller than "tool permissions", so the group goes next.
3. Then interview loop, then offer decision, then tool permissions.
4. Agent prompt goes last. It has the smallest id of all, but it needs tool permissions.

The path:

1. job spec format
2. {candidate dossier, scoring rubric}, read together
3. interview loop
4. offer decision
5. tool permissions
6. agent prompt

Tool permissions landing near the end looks odd at first. It's only there because of the alphabetical tie-break; any order that respects the prerequisites would be correct. What matters is that it's the **same** order every time, whatever order the docs came in. That's what keeps the prompt beginning identical for the prefix cache. If you want a different order among ready docs, change the priority (a weight per doc, for example) and keep it deterministic.

---

## 7. Why the DFS has no recursion

The usual Tarjan is recursive. Each recursive call is a stack frame on the thread stack, so a chain of a million docs means a million frames, and .NET's default stack is about 1 MB. That crashes.

The code keeps its own stack instead:

- `dfs[]` is the call stack: the top is the node being worked on.
- `cursor[v]` remembers which edge of `v` to look at next. That's the part of a recursive frame you actually need.
- When a node finishes, its parent takes its low-link. That's the line that runs after a recursive call returns.

The tests run a 1,000,000-doc chain and a 1,000,000-doc ring on a thread with a 256 KB stack. Any recursion would overflow there.

---

## 8. Tarjan vs Kosaraju

Kosaraju is the other standard way to find SCCs. It runs two DFS passes: one on the graph to record finish order, then one on the **reversed** graph, starting from the node that finished last. Each walk in the second pass collects

one group.

| | Tarjan | Kosaraju |
|---|---|---|
| Passes | one DFS | two DFS, plus building the reversed graph |
| Extra memory | index, low-link, component stack | a full reversed copy of the edges |
| Group order | reverse topological | topological |
| Easier to explain | low-link takes a bit to sink in | two plain DFS passes |

Both are O(V + E). I measured them on my laptop to see what the constant factors look like.

Random graphs with a quarter as many nodes as edges, where 2% of edges point backwards, so a lot of them close cycles. My laptop, Intel Core Ultra 5 225U, .NET 10, BenchmarkDotNet's default job:

| Edges | Tarjan | Kosaraju | Kosaraju / Tarjan |
|---|---|---|---|
| 10,000 | 73 us | 147 us | 2.0x |
| 100,000 | 1.7 ms | 2.3 ms | 1.4x |
| 1,000,000 | 23.3 ms | 28.0 ms | 1.2x |

The full table is in `benchmarks/RESULTS.md`. The gap shrinks as the graph grows. These are one laptop's numbers on random graphs, so read them as the ratio between the two, not as what you'd get on your own docs.

Kosaraju also allocates about 1.64x as much memory, because of the reversed graph. Both give exactly the same groups; a test checks that on 1,000 random graphs.

---

## 9. Complexity

| Step | Time | Memory |
|---|---|---|
| Tarjan | O(V + E) | O(V) on top of the graph |
| Kosaraju | O(V + E) | O(V + E) for the reversed graph |
| Condensation | O(V + E) | O(V + E) |
| Kahn's with a tie-break | O(V + E + V log V) | O(V) |

The `V log V` is the priority queue that picks the smallest ready group. Without a tie-break, a plain queue makes Kahn's O(V + E).

---

## 10. The tests, named after what they protect

| Test | Behaviour |
|---|---|
| `Chain_is_read_prerequisite_first` / `Diamond_breaks_ties_by_id` | the basic order and the tie-break |
| `One_big_cycle_is_one_group` / `Two_cycles_linked` / `Two_cycles_linked_the_other_way` | cycles become groups, and groups are ordered |
| `Doc_after_a_cycle_waits_for_the_whole_cycle` | a doc that needs one member of a cycle waits for the whole group |

| Test | Behaviour |
|---|---|
| `Self_loop_is_a_cycle_of_one` | a doc that lists itself |
| `Input_order_does_not_change_the_path` | the stable order the prefix cache needs |
| `Rubric_and_dossier_are_read_together` / `Full_reading_order` | the hiring assistant's pile from section 4 |
| `Unknown_dependency_is_reported_with_its_doc` / `Duplicate_doc_id_is_rejected` / `Duplicate_edges_count_once` | input checks |
| `Tarjan_matches_reachability_and_Kosaraju` | 1,000 random graphs: same group exactly when each node reaches the other (checked with a slow BFS), same groups from Kosaraju, and every edge between groups points forward in the order |
| `Plans_from_both_algorithms_are_identical` | 1,000 more random graphs: Tarjan and Kosaraju give the same path |
| `Every_prerequisite_comes_earlier_or_in_the_same_group` | the promise the path makes |
| `Million_node_chain_does_not_overflow` / `Million_node_ring_is_one_component` | no recursion, on a 256 KB stack |

## 11. Going further

**"New docs arrive every day."** Rerunning everything is O(V + E) and usually fast enough (see the benchmark). Keeping SCCs up to date edge by edge is a much harder problem; I'd only go there if a full rerun gets too slow.

**"Some docs matter more."** Swap the tie-break: priority by a weight, by how often a doc is used, or by a team's preference. Keep it deterministic, or the prompt beginning changes between calls.

**"The group is too big to read together."** A cycle of twenty docs isn't a useful reading step. The group is still correct; what helps is reporting it, so someone can break the circular references in the docs themselves.

**"The context window can't fit every doc."** Take groups in order until the budget runs out. Never cut a group in half: if you include one doc of a cycle, include the rest of it. Choosing *which* docs to include under a budget is a separate problem (a knapsack), and it's coming later in this repo.

## 12. Common mistakes

| Mistake | What goes wrong | Better |
|---|---|---|
| Run Kahn's on the raw graph | Every doc on a cycle, and everything after it, is silently left out | Find SCCs first, order the condensation |
| Recursive DFS | Stack overflow on a long chain | Keep your own stack and a cursor per node |
| Update low-link from every visited node | Edges into a finished group pull nodes into the wrong group | Only use nodes still on the component stack |
| Forget the self-loop | A doc that lists itself looks like an ordinary doc | Mark it as a cycle of one |
| Count a duplicate edge twice | In-degrees never reach 0 in Kahn's | Remove duplicate edges when building the graph |

| Mistake | What goes wrong | Better |
|---|---|---|
| Tie-break by input order | Same docs, different order, different prompt | Tie-break by something in the docs themselves |
| Edges the wrong way round | The path comes out backwards | Decide once: prerequisite to the doc that needs it |

## 13. Run it

```
cd dsa/graphs/learning-path-from-docs
dotnet test                                              # 65 tests
cd benchmarks/LearningPath.Benchmarks
dotnet run -c Release -- --filter *                      # Tarjan vs Kosaraju
```

Requires .NET 10 SDK.

## Sources


- Tarjan, R. E. (1972), "Depth-first search and linear graph algorithms", SIAM Journal on Computing 1(2)
- CMake: `cmComputeTargetDepends.cxx`; `cmComputeComponentGraph.h`
- LLVM: `SCCIterator.h`
- Haskell `containers`: `Data.Graph`


*Details come from public source code and docs as of October 2026. Implementations change; read them as "publicly described as...".*